Módulo 02: Deploy Containerized Web Applications to Azure App Service

PaaS de contenedores en Linux, autenticación Zero-Trust con Identidad Administrada y persistencia en Azure App Service

Alineación Curricular Oficial Microsoft Learn (Curso AI-200T00-A • Examen AI-200): Dominio 1: Contenedores (20% – 25%)

Cubre las competencias oficiales del **Módulo 2: Implementación de contenedores en Azure App Service** (5 días / 120 hrs). **Objetivos Oficiales (Slide 14):** (1) Implementar contenedores Linux desde ACR y registros Docker V2; (2) Configurar runtime, Startup commands, puertos y persistencia; (3) Administrar App Settings, cadenas de conexión, configuraciones de ranura y referencias a Key Vault; (4) Observabilidad y diagnóstico con Log stream, Kudu SCM y Azure Monitor (KQL). Consulte la Guía de Estudio Oficial AI-200 y el Ejercicio Oficial de Despliegue en App Service (30 min).

PRERREQUISITOS UNIVERSALES DEL ENTORNO

- **Suscripción de Azure:** Requiere plan con soporte de App Service Linux dedicado (ej. `B1` o `P0v3`).
- **Azure CLI:** Versión 2.50+ instalada localmente o en Azure Cloud Shell (`az --version`).
- **Python 3.10+:** Para ejecutar scripts de despliegue automatizado de infraestructura (`azdeploy.py`).
- **Permisos RBAC:** Roles `Contributor` y `User Access Administrator` u `Owner` para asignación de roles.
- **Sin Docker local:** No se requiere Docker Desktop; la imagen se compila en ACR Tasks y se corre en PaaS.

AVISOS CRÍTICOS Y CONSIDERACIONES DE AZURE

- **DNS Global Único:** Los nombres de ACR (`*.azurecr.io`) y Web Apps (`*.azurewebsites.net`) son públicos y únicos.
- **Propagación de Roles:** Las asignaciones de roles RBAC en Entra ID tardan típicamente entre 60 y 120 segundos.
- **Sintaxis de Consola:** Se documenta sintaxis dual: PowerShell (acento grave `` ` ``) y Bash (barra invertida `\`).
- **Almacenamiento Efímero:** Por defecto, los contenedores Linux en App Service no persisten archivos sin configurar `/home`.

Estructura del Proyecto Fuente: Paquete oficial ubicado en `CODE/2. app-svc-container-python/` (descargable desde app-svc-container-python.zip) que contiene el script de infraestructura (`azdeploy.py`), el servicio Flask en `api/` (con `Dockerfile`, `requirements.txt` y `main.py`) y el documento de prueba `document.txt`.

Didáctica de Comandos Azure CLI: Jerarquía y Parámetros del Módulo

Estructura jerárquica de comandos: En este módulo profundizamos en `az appservice` y `az webapp`. Toda instrucción sigue el estándar: `az <servicio> <subgrupo> <acción> [--parámetros]`. A continuación los parámetros cardinales para orquestar contenedores PaaS seguros con RBAC:

Parámetro / Flag	Propósito Técnico en App Service & ACR	Ejemplo en este Laboratorio
`az webapp create`	Aprovisiona el contenedor web PaaS en un plan Linux dedicado.	`az webapp create --plan $APP_PLAN ...`
`az webapp identity assign`	Crea un Service Principal administrado por Azure en Microsoft Entra ID.	`az webapp identity assign -g $RG -n $APP_NAME`
`--role AcrPull`	Rol RBAC mínimo para autenticar y extraer capas de contenedor desde ACR.	`--role "AcrPull" --scope $ACR_ID`
`--acr-use-identity true`	Instruye a App Service a usar el token MSI en lugar de contraseñas de admin.	`az webapp config set --acr-use-identity true`
`--settings`	Inyecta variables de entorno en el contenedor (Application Settings).	`--settings WEBSITES_ENABLE_APP_SERVICE_STORAGE=true`
`--docker-container-logging`	Habilita el enrutamiento de stdout/stderr hacia el sistema de archivos.	`az webapp log config --docker-container-logging filesystem`

Fase 1 (Task 1): Autenticación, Suscripción y Proveedores de Recursos

`az login & az provider register (Microsoft.Web & Microsoft.ContainerRegistry)`

Identidad & API

Inicie sesión en Azure, seleccione su suscripción y registre los proveedores de recursos obligatorios:

```
# 1. Autenticación interactiva en la suscripción (seleccione su cuenta y suscripción)
az login

# 2. Verificar la suscripción activa actual
az account show --output table

# 3. Registrar los Resource Providers obligatorios para App Service y Container Registry
az provider register --namespace Microsoft.ContainerRegistry
az provider register --namespace Microsoft.Web

# 4. Verificar el estado del proveedor de App Service (debe figurar "Registered")
az provider show --namespace Microsoft.Web --query "registrationState" -o tsv
```

¿Por qué registrar Microsoft.Web y Microsoft.ContainerRegistry?: Son los proveedores ARM responsables del aprovisionamiento de App Service Plans, Web Apps for Containers, certificados SSL y registros OCI en Azure.

INSPECCIÓN Y GESTIÓN DE PROVEEDORES EN AZURE PORTAL (GUI)

Navegación: Busque **Subscriptions** en la barra superior -> Seleccione su suscripción -> En el menú izquierdo bajo **Settings**, haga clic en **Resource providers** -> Filtre por `Microsoft.Web` y verifique que figure en estado **Registered** (si está en *NotRegistered*, selecciónelo y pulse **Register**).

FUNDAMENTO DE AZURE APP SERVICE: DUALIDAD WEB APP (CARGA LÓGICA) VS APP SERVICE PLAN (CÓMPUTO FÍSICO)

- **Modelo PaaS Totalmente Administrado:** Abstracción completa de infraestructura. Azure gestiona el aprovisionamiento de VMs, parches del kernel de Linux, balanceo de carga L7 y terminación TLS/SSL sin sobrecarga operativa.
- **Web App (Carga Lógica y Runtime):** Entorno donde reside la aplicación (código o contenedor OCI). Encapsula configuraciones (App Settings), variables de entorno, certificados SSL, dominios personalizados y ranuras de despliegue (*deployment slots*).
- **App Service Plan (Cómputo Físico y Costos):** Representa la granja dedicada de servidores virtuales subyacente. Define la región geográfica, cantidad de núcleos vCPU, memoria RAM disponible, límites de escalado elástico y el nivel de facturación (SKU).
- **Densidad y Aislamiento Multitenant:** Múltiples Web Apps pueden cohabitar en un mismo App Service Plan compartiendo recursos para optimizar costos, o aislarse en planes dedicados para garantizar rendimiento predecible en producción.

Desglose Técnico del Código del Laboratorio

Análisis arquitectónico de azdeploy.py, Dockerfile y la API de procesamiento (CODE/2. app-svc-container-python)

Análisis Arquitectónico de los Archivos del Proyecto

1. azdeploy.py (Automatización de Infraestructura)

INFRAESTRUCTURA COMO CÓDIGO

Script en Python que orquesta el aprovisionamiento de recursos en Azure sin requerir plantillas Bicep ni Terraform:

- **Variables de entrada (L8-54):** Define `rg` (Resource Group), `location` (región, ej. `canadacentral` o `eastus`) y `sku` (cómputo, por defecto `P0v3` o alternativo `B1`). Son los parámetros configurables de usuario.
- **Ajuste de SKU de Cómputo:** Incluye especificaciones arquitectónicas de tiers (Basic, Standard, Premium v3 e Isolated v2). Por defecto utiliza `sku = "P0v3"` (Premium v3 de entrada, 1 vCPU / 4 GB RAM) o `sku = "B1"` (Basic para suscripciones con cuota ajustada).
- **Identidad y Hashing único (L121-128):** Obtiene el Object ID de Entra ID mediante `az ad signed-in-user show`, calcula un hash SHA-1 de 8 caracteres y deriva los nombres `acr{user_hash}`, `plan-docprocessor-{user_hash}` y `app-docprocessor-{user_hash}`, garantizando unicidad global de FQDN.
- **Idempotencia en Recursos (L147-233):** Comprueba la existencia previa de Resource Group, ACR y App Service Plan antes de crearlos.
- **Exportación Multi-Shell (L88-120):** Escribe `.env` (para Bash: `export VAR="..."`) y `.env.ps1` (para PowerShell: `$env:VAR = "..."`) con saltos LF para interoperabilidad universal.

2. api/Dockerfile (Definición de Contenedor OCI)

CONTENEDOR OCI

Especificación de capas optimizada para ejecución de microservicios en Azure App Service sobre Linux:

- `FROM python:3.11-slim`: Imagen oficial reducida basada en Debian (~150 MB), minimizando la superficie de ataque y el tiempo de extracción de capas.
- `COPY requirements.txt . && RUN pip install --no-cache-dir -r requirements.txt`: Cacheo de dependencias de Flask y Gunicorn sin almacenar tarballs temporales de pip.
- `RUN mkdir -p /home/processed`: Prepara el directorio de almacenamiento local (en App Service se montará sobre volumen CIFS compartido).
- `EXPOSE 80`: Declara el puerto de escucha que el proxy reverso frontal de App Service detecta automáticamente.
- `CMD ["gunicorn", "--bind", "0.0.0.0:80", "--workers", "2", "--access-logfile", "-", "main:app"]`: Arranca el servidor WSGI en formato exec (JSON array) para propagación directa de señales SIGTERM y logging a stdout.

3. api/main.py (Microservicio Flask de Procesamiento de Documentos)

MICROSERVICIO REST

Servidor REST que simula inferencia de IA (extracción de entidades, palabras clave, análisis de sentimiento) y persistencia:

- **Configuración Twelve-Factor (L13-21):** Ingesta variables operativas mediante `os.getenv()` (`MAX_DOCUMENT_SIZE_MB`, `STORAGE_PATH`, `ENVIRONMENT`).
- **Ruta `/health`:** Health check probe ligero para los sondeos de disponibilidad (Liveness Probe) del balanceador PaaS de App Service.
- **Ruta `/process`:** Ingiere texto o JSON, valida límites de tamaño (413), calcula métricas y persiste el resultado en `STORAGE_PATH/{doc_id}.json`.
- **Ruta `/documents`:** Inspecciona el volumen montado y expone la lista de documentos analizados para auditoría.

4. api/requirements.txt (Dependencias del Microservicio de IA)

LIBRERÍAS PYTHON

Lista estricta de dependencias sin fijación de build-tools para evitar almacenamiento redundante en capas OCI:

- `Flask==3.0.0`: Framework WSGI ligero para la API REST de procesamiento y controladores de endpoints.
- `gunicorn==21.2.0`: Servidor HTTP WSGI para UNIX de producción con soporte de múltiples workers concurrentes.
- `werkzeug==3.0.1`: Utilidades WSGI subyacentes para parseo seguro de solicitudes, cabeceras y manejo de payloads.

Preparación del Archivo de Despliegue en VS Code (Task 1)

Configurar azdeploy.py (Variables de Configuración)

Configuración

Abra `azdeploy.py` en VS Code y configure sus parámetros antes de ejecutar el menú:

```
# En azdeploy.py edite las variables superiores de configuración:
rg = "rg-docprocessor-lab"        # Su grupo de recursos
location = "canadacentral"          # Región recomendada (o eastus)
sku = "P0v3"                    # Premium v3 por defecto (o "B1" para ahorro)
```

Notas Oficiales de Preparación (Task 1):

- **Estructura y Ubicación:** El código fuente reside en `CODE/2. app-svc-container-python/` (descargable desde `app-svc-container-python.zip`). Abra la carpeta en VS Code (*File > Open Folder...*) aceptando *"Yes, I trust the authors"*.
- **Región y SKU:** Ingrese `canadacentral` (o `eastus`) y verifique el `sku` configurado (`P0v3` por defecto; use `B1` si su cuenta tiene restricciones de cuota). Guarde con *File > Save*. No modifique ninguna otra línea.
- **Apertura de Terminal:** En el menú superior seleccione *Terminal > New Terminal* (si no está visible, use `... > Terminal > New Terminal`).

CONTINUIDAD ARQUITECTÓNICA: LECCIONES Y CONEXIÓN CON EL MÓDULO 1 (ACR & COSMOS DB - SLIDE 12 OFICIAL)

- **Jerarquía OCI de ACR:** Registro > Repositorio > Artefacto con soporte de namespaces para gobernanza modular corporativa.
- **Etiquetas vs Resúmenes (Digests):** Las etiquetas (`:v1`) son referencias mutables; los resúmenes SHA-256 son inmutables para garantizar reproducibilidad estricta en producción.
- **Bloqueo e Inmutabilidad:** Evite eliminaciones o sobreescrituras accidentales en ACR ejecutando `az acr repository update --image <img> --write-enabled false`.
- **ACR Tasks & Ciclo de Vida:** Compilaciones serverless en la nube, triggers automáticos por commit e imagen base, y purga programada de manifiestos sin etiquetar (*untagged*).
- **Desacoplamiento con Cosmos DB:** Reutilice `CosmosClient` como singleton para pool de conexiones, diseñe la Partition Key como decisión inmutable y priorice lecturas puntuales sobre consultas para ahorrar RUs.

Creación de App Service y Plan: Azure Portal (GUI) vs. Azure CLI

Análisis técnico de opciones del asistente: Basics, Cómputo Linux, Contenedores OCI, Redes y Comandos CLI

1. Asistente Gráfico del App Service Plan: Dimensionamiento y Cómputo Linux

Azure Portal › App Service Plans › + Create

IDENTIDAD Y SISTEMA OPERATIVO

- **Resource Group:** `rg-docprocessor-lab` (limita el ciclo de vida, políticas de gobernanza RBAC y auditoría de costos).
- **Operating System: Linux** (MANDATORIO: establece `reserved: true` en ARM para aprovisionar workers dedicados de Linux para contenedores OCI).
- **Region:** `East US 2` (co-ubicado en la misma región que el registro ACR para latencia sub-milisegundo y costo de transferencia cero).

TIERS DE CÓMPUTO (SKU)

- **Free / Shared (F1/D1):** Cómputo multi-tenant básico sin soporte para contenedores Docker/OCI.
- **Basic (B1):** 1 vCPU, 1.75 GB RAM. Nivel dedicado mínimo para desarrollo y pruebas de APIs de inferencia.
- **Premium v3 (P0v3 / P1v3):** Nivel productivo para IA con procesadores rápidos, escalado de hasta 30 nodos, redundancia de zonas y soporte nativo de VNet Integration.

ARQUITECTURA Y FABRIC CONTROLLER

- **Worker Pool Dedicado:** El Fabric Controller de Azure reserva instancias virtuales dedicadas que no comparten CPU ni memoria con otros tenants.
- **Densidad de Apps:** Múltiples Web Apps pueden coexistir en el mismo Plan de Hospedaje optimizando la inversión en hardware.
- **Escalabilidad:** Soporta *Scale Up* (aumento de vCPU/RAM sin downtime) y *Scale Out* (adición horizontal de réplicas en cluster).

2. Asistente Gráfico de Web App for Containers: Pestañas "Basics" y "Container"

Azure Portal › App Services › + Create › Web App

Basics | Database | Container | Networking | Monitoring | Tags | Review + create

PESTAÑA "BASICS": IDENTIDAD Y ALOJAMIENTO

- **Name:** FQDN global único `<app-name>.azurewebsites.net` (debe ser alfanumérico, globalmente irrepetible).
- **Publish:** Seleccione **Container** (le indica a la plataforma PaaS que orqueste la ejecución de una imagen OCI en lugar de runtime de código).
- **Operating System: Linux** (se bloquea automáticamente para alinearse con el Plan asociado).
- **App Service Plan:** Seleccione el plan Linux previamente creado (`plan-docprocessor-...`) que dictará los recursos de CPU y memoria.

PESTAÑA "CONTAINER": ORIGEN OCI Y EJECUCIÓN

- **Image Source: Azure Container Registry** (admite también Docker Hub, GHCR y registros privados con autenticación).
- **Registry, Image & Tag:** Seleccione su ACR, el repositorio `docprocessor` y el tag inmutable `v1` .
- **Startup Command:** Opcional. Permite sobrescribir la directiva `CMD` del Dockerfile sin reconstruir la imagen ni invalidar el `ENTRYPOINT` .
- **Terminación TLS:** El balanceador PaaS frontal gestiona y termina certificados TLS 1.3 de forma nativa.

3. Equivalencia en Azure CLI: Modos de Aprovisionamiento (Plan y Web App)

`az appservice plan create & az webapp create` — CLI vs Portal

Comandos oficiales de Azure CLI que replican exactamente los parámetros configurados en el asistente gráfico del Portal:

```
# 1. Crear el App Service Plan Linux dedicado (flag --is-linux mandatorio)
az appservice plan create `
  --resource-group $env:RESOURCE_GROUP `
  --name $env:APP_PLAN `
  --sku B1 `
  --is-linux

# 2. Crear la Web App configurada para extraer desde el Registro ACR privado
az webapp create `
  --resource-group $env:RESOURCE_GROUP `
  --plan $env:APP_PLAN `
  --name $env:APP_NAME `
  --container-image-name "$($env:ACR_NAME).azurecr.io/docprocessor:v1"
```

Regla de Arquitectura PaaS: En el Portal gráfico, la selección de *Publish: Container* activa el motor de despliegue OCI. En Azure CLI, el parámetro `--container-image-name` habilita automáticamente el modo contenedor. Para entornos empresariales con Zero-Trust, en lugar de credenciales maestras de administrador, en las siguientes fases habilitaremos **Managed Identity** (`az webapp identity assign`).

Aprovisionamiento de Servicios Base con azdeploy.py

Compilación serverless en ACR Tasks, creación del App Service Plan Linux y carga de variables de sesión

ORQUESTACIÓN AUTOMATIZADA CON AZDEPLOY.PY (TASK 1)

El script interactivo `azdeploy.py` ejecuta la creación del Resource Group, aprovisiona el ACR privado con autenticación de administrador deshabilitada (Zero-Trust), envía el contexto de `api/` a compilar de forma remota en ACR Tasks y crea el App Service Plan en Linux.

Paso 2.1: Crear Azure Container Registry y Compilar la Imagen (Opción 1)

`python azdeploy.py (Opción 1: ACR & Build)`

ACR Tasks Serverless

Abra la terminal integrada en VS Code (*Terminal > New Terminal*) y ejecute la opción 1:

```
# 1. Ejecutar el script interactivo desde la raíz del proyecto (CODE/2. app-svc-container-python)
python azdeploy.py

# En el menú presentado, seleccione la Opción 1:
# 1. Create Azure Container Registry and build container image
Checking/creating resource group 'rg-docprocessor-lab'...
Creating Azure Container Registry 'acra1b2c3d4'...
ACR created: acra1b2c3d4. Login server: acra1b2c3d4.azurecr.io
Building and pushing container image to ACR... This may take a few minutes...
Image built and pushed: acra1b2c3d4.azurecr.io/docprocessor:v1
```

Compilación serverless en la nube: La opción 1 invoca `az acr build` empaquetando `api/Dockerfile` y `api/main.py` en un tarball enviado por TLS a ACR Tasks, entregando la imagen `docprocessor:v1` lista sin depender de Docker local.

Paso 2.2: Crear el App Service Plan Linux y Cargar Variables de Entorno (Opciones 2 y 4)

`python azdeploy.py (Opción 2: App Service Plan & Opción 4: Salir)`

Cómputo Linux PaaS

Seleccione la opción 2 para aprovisionar el plan de proceso dedicado y exportar las variables de entorno:

```
# 2. En el menú de azdeploy.py, seleccione la Opción 2:
# 2. Create App Service Plan
Creating App Service Plan 'plan-docprocessor-a1b2c3d4'...
App Service Plan created: plan-docprocessor-a1b2c3d4 (SKU: B1 o P0v3)
Environment variables saved to: .env and .env.ps1

# 3. Salir del script seleccionando la Opción 4: Exit

# 4. Cargar las variables en su sesión de terminal activa:
# En Windows PowerShell:
. .\.env.ps1
# 📝 Nota: Mantenga esta terminal abierta durante todo el ejercicio.

# En Bash / macOS / Linux:
source .env
```

NIVELES DE SKU DEL APP SERVICE PLAN

- **Basic (B1):** 1 núcleo vCPU, 1.75 GB RAM. Nivel mínimo dedicado para desarrollo y pruebas de contenedores Linux.
- **Premium v3 (P0v3 / P1v3):** Diseñado para producción de IA con procesadores rápidos, escalado automático hasta 30 réplicas y soporte de VNet Integration.
- **Flag `--is-linux` (`reserved: true`):** Asigna nodos Linux dedicados para ejecutar contenedores OCI de forma nativa.

CREACIÓN E INSPECCIÓN EN AZURE PORTAL (GUI)

Azure Portal › Resource groups › rg-docprocessor-lab

- **1. Navegación al Grupo de Recursos:** En la barra de búsqueda principal escriba **Resource groups** y seleccione `rg-docprocessor-lab`.
- **2. Validación de ACR:** Pulse sobre el registro `acra1b2c3d4`. En el menú lateral navegue a **Repositories** y confirme que la imagen `docprocessor` esté listada con la etiqueta inmutable `v1`.
- **3. Validación del Plan Linux:** Regrese al grupo de recursos y seleccione `plan-docprocessor-...`. En la sección **Overview** confirme: **OS: Linux**, **Pricing tier: B1** y **Apps: 0** (esperando la vinculación de la Web App).

Creación de Web App for Containers y Detección de Puertos

Aprovisionamiento con az webapp create, proxy inverso PaaS y configuración de arranque (Task 2)

Paso 3.1 (Task 2): Crear la Web App configurada para extraer desde el Registro ACR

az webapp create (--container-image-name)

PaaS Container Instance

Cree el recurso Web App en el plan de hospedaje Linux indicando la imagen compilada previamente en ACR:

```
# Comando multilínea en PowerShell ( ` )
az webapp create `
  --resource-group $env:RESOURCE_GROUP `
  --plan $env:APP_PLAN `
  --name $env:APP_NAME `
  --container-image-name "$($env:ACR_NAME).azurecr.io/docprocessor:v1"

# Equivalente en Bash / Linux:
# az webapp create --resource-group $RESOURCE_GROUP --plan $APP_PLAN \
#   --name $APP_NAME --container-image-name "$ACR_NAME.azurecr.io/docprocessor:v1"

# Consultar el estado de aprovisionamiento de la Web App
az webapp show -g $env:RESOURCE_GROUP -n $env:APP_NAME --query "{State:state, Host:defaultHostName}" -o table
```

Nota Oficial de Arquitectura: De forma predeterminada, Azure Container Registry es privado. App Service necesita una forma de autenticarse ante ACR antes de poder extraer la imagen. En el siguiente paso configuraremos una *System-Assigned Managed Identity* en lugar de almacenar credenciales de administrador en la configuración.

MAPEO DE PUERTOS Y ENRUTAMIENTO INTERNO

- **Detección automática:** App Service analiza el contenedor y detecta la cláusula `EXPOSE 80` del `Dockerfile`.
- **Proxy inverso frontal:** El balanceador PaaS recibe solicitudes públicas en los puertos 80/443 (HTTPS) y las redirige internamente al puerto en el que escucha Gunicorn (puerto 80).
- **Variable `WEBSITES_PORT`:** Si la aplicación escucha en un puerto personalizado (ej. 5000 o 8080), es obligatorio definir `WEBSITES_PORT=80` en Application Settings.
- **Enlace a `0.0.0.0`:** En contenedores Docker, la aplicación debe enlazar a `0.0.0.0` (no a `127.0.0.1`), de lo contrario el proxy de App Service no podrá alcanzar el servicio.

TIMEOUT DE INICIO DEL CONTENEDOR

- **Límite predeterminado:** App Service espera hasta **230 segundos** a que el contenedor responda a las sondas HTTP iniciales antes de abortar el arranque.
- **Modelos pesados de IA:** Si la aplicación descarga pesos de modelos o bibliotecas pesadas durante el arranque, este tiempo puede ser insuficiente.
- **Variable de ampliación:** Se debe configurar `WEBSITES_CONTAINER_START_TIME_LIMIT=600` (configurable hasta un máximo de 1800 segundos / 30 minutos).
- **Diagnóstico de fallos:** Si el contenedor se reinicia en bucle (crash loop), revise los logs del motor Docker en Kudu.

Paso a Paso en Azure Portal (GUI): Creación y Validación de la Web App

Azure Portal › App Services › + Create › Web App

1. Basics | 2. Container | 3. Networking | 4. Monitoring | 5. Tags | 6. Review + create

- **Pestaña Basics:** Seleccione Suscripción y Resource Group `rg-az200-containers`. Ingrese Nombre (DNS global: `<app>.azurewebsites.net`). Configure **Publish: Container**, **Operating System: Linux**, Región `East US` y asocie el **Linux Plan** creado previamente.
- **Pestaña Container:** Active **Image Source: Azure Container Registry**. Seleccione Registro (`cr...`), Repositorio (`docprocessor`) y Tag (`v1`). Deje los comandos de inicio predeterminados.
- **Inspección y Estado:** Tras pulsar **Review + create** y completar el despliegue, abra la hoja **Overview** para confirmar **Status: Running** y copie el **Default domain** (HTTPS). En **Deployment Center** valide el registro OCI configurado.

COMANDO DE INICIO (STARTUP COMMAND) VS DOCKERFILE CMD

- **Invalida únicamente CMD:** En App Service Linux, el comando de inicio configurado anula la directiva `CMD` del Dockerfile, manteniendo la directiva `ENTRYPOINT` intacta.
- **Casos de uso en IA:** Utilícelo para pasar argumentos en runtime, ejecutar scripts de migraciones (ej. `python manage.py migrate`) o inicializaciones antes del servidor WSGI.
- **Configuración Azure CLI:** `az webapp config set -g $RG -n $APP_NAME --startup-file "gunicorn --bind 0.0.0.0:80 api.main:app"`.

ORÍGENES DE IMAGEN, TLS Y WEBHOOKS DE ACR

- **Orígenes OCI V2:** Admite Azure Container Registry (recomendado para producción con Managed Identity), Docker Hub, GitHub Container Registry (GHCR) y registros privados.
- **Terminación TLS Administrada:** App Service gestiona certificados y termina TLS 1.3 en puertos 80/443; el contenedor recibe HTTP plano en `WEBSITES_PORT` sin sobrecarga SSL interna.
- **Despliegue Continuo (CI/CD):** Configure webhooks en ACR (`az acr webhook create`) para desencadenar el redeploy automático de la Web App ante cada nuevo push de imagen.

Identidad Administrada, Rol AcrPull y Configuración Runtime

Extracción passwordless de imágenes, asignación RBAC en Entra ID y persistencia en /home (Tasks 2 y 3)

Principio de Mínimos Privilegios (Zero-Trust): Las identidades administradas (MSI) respaldadas por Microsoft Entra ID eliminan credenciales maestras en texto plano. App Service solicita tokens RBAC en runtime a Entra ID para descargar la imagen OCI desde el registro ACR privado de forma transparente.

1. Automatización con Azure CLI: Identidad, Contenedor, Always On y Logs (Tasks 2 y 3)

`az webapp identity & role assignment`

Task 2 • Zero-Trust

Habilite MSI asignada por el sistema y asigne rol `AcrPull` en ACR:

```
# 1. Habilitar Managed Identity en la Web App
az webapp identity assign -g $env:RESOURCE_GROUP -n $env:APP_NAME

# 2. Obtener Principal ID de Web App e ID del ACR
$PRINCIPAL_ID = az webapp identity show -g $env:RESOURCE_GROUP `
  -n $env:APP_NAME --query principalId -o tsv
$ACR_ID = az acr show -g $env:RESOURCE_GROUP -n $env:ACR_NAME `
  --query id -o tsv

# 3. Asignar rol AcrPull en el ámbito del registro
az role assignment create --assignee $PRINCIPAL_ID `
  --scope $ACR_ID --role AcrPull
```

`az webapp config (container, settings, logs)`

Task 3 • Runtime & Logs

Enlace imagen OCI, active Always On y logging en filesystem:

```
# 1. Configurar Kudu para extraer usando la MSI
az webapp config set -g $env:RESOURCE_GROUP -n $env:APP_NAME `
  --acr-use-identity true --acr-identity [system]
az webapp config container set -g $env:RESOURCE_GROUP `
  -n $env:APP_NAME --container-image-name "$($env:ACR_NAME).azurec
r.io/docprocessor:v1" `
  --container-registry-url "https://$($env:ACR_NAME).azurecr.io"

# 2. Configurar puerto 80 y almacenamiento CIFS en /home
az webapp config appsettings set -g $env:RESOURCE_GROUP `
  -n $env:APP_NAME --settings WEBSITES_PORT=80 WEBSITES_ENABLE_APP_
SERVICE_STORAGE=true

# 3. Habilitar Always-On y logging Docker en filesystem
az webapp config set -g $env:RESOURCE_GROUP -n $env:APP_NAME --alwa
ys-on true
az webapp log config -g $env:RESOURCE_GROUP -n $env:APP_NAME --dock
er-container-logging filesystem
```

2. Configuración Paso a Paso en Azure Portal (GUI): Identidad, ACR, Runtime, Always On y Logs

BLADE 1 & 2: MANAGED IDENTITY Y ROL ACRPULL EN ACR

App Services › <app> › Settings › Identity

System assigned | User assigned

- **Paso 1 (Habilitar Managed Identity):**
  - *System assigned* › Status en **On** › Clic **Save** (registra el Service Principal en Microsoft Entra ID y genera el `Object ID`).

Container registries › <acr> › Access control (IAM)

- **Paso 2 (Asignar Rol AcrPull en ACR):**
  - Clic **+ Add** › **Add role assignment** › Rol: **AcrPull** › Asignar acceso a: **Managed identity** (Web App) › **Review + assign**.

App Services › <app> › Deployment › Deployment Center

- **Paso 3 (Extracción OCI Passwordless):**
  - *Source*: Container Registry › *Auth*: **Managed Identity** › *Image*: `docprocessor:v1` › Clic **Save**.

BLADE 3: CONFIGURATION (PUERTO, /HOME Y ALWAYS ON)

App Services › <app> › Settings › Configuration

Application settings | General settings | Path mappings

- **Pestaña Application settings (+ New setting):**
  - `WEBSITES_PORT` = `80` *(Enruta el tráfico entrante al puerto expuesto en el contenedor).*
  - `WEBSITES_ENABLE_APP_SERVICE_STORAGE` = `true` *(Monta SMB/CIFS persistente en `/home`).*
- **Pestaña General settings:**
  - **Always on:** Switch en **On** *(Mantiene el worker en RAM; previene idle timeout y cold starts de IA).*
  - **Health check:** Habilite y defina Path en `/health` *(Sondeo balanceado PaaS cada 1 min).*
- Clic en el botón superior **Save** › Confirme en **Continue** para reciclar el proceso.

BLADE 4: APP SERVICE LOGS Y TRANSMISIÓN EN TIEMPO REAL

App Services › <app> › Monitoring › App Service logs

- **Configuración de Logs en Disco PaaS:**
  - *Application logging*: Seleccione **File System** (captura stdout/stderr del contenedor OCI).
  - *Quota (MB)*: `35` (límite rotativo de disco) | *Detailed error messages*: **On**.

App Services › <app> › Monitoring › Log stream

- **Transmisión en Vivo (Log Stream):**
  - Visualización streaming de pull OCI (`Pulling image...`), inicio de Gunicorn y probes `GET /health 200 OK` en vivo (equivalente a `az webapp log tail`).

App Services › <app> › Diagnose and solve problems › Container Issues

- **Diagnóstico de Crash y Startup Timeout:**
  - Detección inmediata de contenedores fallidos (código de salida distinto de 0, timeouts OCI y fallas de memoria OOM).

BLADE 5: DEPLOYMENT SLOTS, SLOT SETTINGS Y SWAP (SLIDES 18 & 21)

App Services › <app> › Settings › Scale up (App Service plan)

- **Prerrequisito de SKU (Scale Up):**
  - Seleccionar tier **Standard S1+** o **Premium** (los tiers *Free F1* y *Basic B1* bloquean la pestaña de Deployment Slots).

App Services › <app> › Deployment › Deployment slots

- **Paso 1 (Creación de Ranura en Panel 'Add a slot'):**
  - Clic **+ Add Slot** › Name: `staging` › *Clone settings from*: producción › Clic **Add** (autogenera FQDN independiente `https://<app>-staging.azurewebsites.net` con 0% de tráfico).

Deployment slots › Clic 'staging' › Settings › Configuration

- **Paso 2 (Anclaje de Variable / Deployment Slot Setting):**
  - Clic **+ New application setting** › Name: `API_ENDPOINT` (o `DATABASE_URL`) › Value: `https://dev.api/v1` › Marcar **[✓] Deployment slot setting** *(Solda la variable al slot; no rota durante el swap)* › Clic **Save**.

App Services › <app> › Deployment slots › Swap

- **Paso 3 (Modal Swap Zero-Downtime & Warm-up):**
  - *Source*: `staging` → *Target*: `production` | *Type*: **Swap**. Azure ejecuta warm-up contra `/health` y conmuta los VIPs de red en el balanceador PaaS sin desconectar usuarios.

Verificación Operativa, Ingesta de Documentos y Consola Kudu

Pruebas REST con PowerShell y curl, streaming de logs y diagnóstico avanzado en SCM (Tasks 4 a 8)

Pasos 5.1 y 5.2 (Tasks 4 y 5): Verificación de Endpoint y Procesamiento de Documento

`Invoke-RestMethod & curl (POST /process & GET /documents)`

API Testing

Obtenga la URL de la aplicación, envíe el archivo document.txt y verifique que se guarde en almacenamiento persistente:

```
# 1. Recuperar el FQDN público de la Web App (Task 4)
$APP_URL = az webapp show --resource-group $env:RESOURCE_GROUP --name $env:APP_NAME --query defaultHostName --output tsv
Write-Host "Application URL: https://$APP_URL"

# 2. Probar la aplicación en Edge abriendo la URL: https://$APP_URL (debe retornar status: running)
# 📝 Nota: La primera petición puede tardar unos segundos mientras se extrae la imagen.

# 3. Enviar el archivo document.txt al endpoint de procesamiento (Task 5 en PowerShell):
$body = Get-Content -Raw -Path "document.txt"
Invoke-RestMethod -Method Post -Uri "https://$APP_URL/process" -ContentType "text/plain" -Body $body | ConvertTo-Json -Depth 10

# Equivalente en Bash / Linux usando curl:
# curl -s -X POST "https://$APP_URL/process" -H "Content-Type: text/plain" --data-binary @document.txt

# 4. Listar los documentos procesados desde el almacenamiento persistente (Task 5):
Invoke-RestMethod -Uri "https://$APP_URL/documents" | ConvertTo-Json -Depth 10
```

Confirmación de persistencia: La respuesta de la API incluye `"storage": {"saved": true, "path": "/home/processed/..."}`. Si el almacenamiento persistente está activo, el archivo permanecerá accesible tras reiniciar la app.

Pasos 5.3 a 5.5 (Tasks 6, 7 y 8): Streaming de Logs, Consola Kudu y Configuración

`az webapp log tail & Kudu SCM Diagnostic Console`

Observabilidad & SCM

Transmita logs en tiempo real, explore la consola SCM (Kudu) y verifique la lista de Application Settings:

```
# 1. Transmitir logs en tiempo real desde la consola (Task 6 - presione Ctrl+C para salir):
az webapp log tail --resource-group $env:RESOURCE_GROUP --name $env:APP_NAME

# 2. Imprimir la URL del sitio Kudu SCM para diagnóstico avanzado (Task 7):
Write-Host "Kudu URL: https://$($env:APP_NAME).scm.azurewebsites.net"

# 3. Listar todas las variables de configuración de la Web App en formato tabla (Task 8):
az webapp config appsettings list --resource-group $env:RESOURCE_GROUP --name $env:APP_NAME --output table
```

NAVEGACIÓN EN LA CONSOLA KUDU (SCM)

Abra `https://<app-name>.scm.azurewebsites.net` en el navegador Edge:

- **Environment:** Muestra variables de entorno Linux, Application Settings inyectadas y tokens MSI de Entra ID.
- **WebSSH:** Terminal de acceso directo al contenedor en el navegador para depuración interactiva de procesos.
- **File Manager:** Navegue a `/home/LogFiles/` para inspeccionar logs de Docker y a `/home/processed/` para ver los JSON persistidos.
- **Process Explorer:** Monitoreo en vivo de subprocesos Gunicorn, hilos de ejecución activos y consumo de memoria RAM.

INSPECCIÓN EN AZURE PORTAL (GUI)

- **Log stream:** En la Web App -> **Monitoring** -> **Log stream**: visualice en vivo las peticiones HTTP y eventos de arranque.
- **Configuration:** En **Settings** -> **Configuration**: confirme que figuren `WEBSITES_PORT=80`, `WEBSITES_ENABLE_APP_SERVICE_STORAGE=true` y `always-on=true`.
- **Advanced Tools:** En **Development Tools** -> **Advanced Tools**: pulse *Go* para acceder a Kudu autenticado.
- **Deployment Center:** Inspeccione el historial de sincronización del webhook con el registro ACR.

VOLCADO DE DIAGNÓSTICO (DIAGNOSTIC DUMP) EN KUDU

- **Descarga ZIP Completa:** Obtenga un volcado integral desde `https://<app>.scm.azurewebsites.net/api/dump` para auditoría forense offline de incidentes en producción.
- **Trazas del Motor Docker:** El archivo ZIP incluye logs históricos de extracción de capas OCI, eventos del daemon de Docker y registros de terminación de workers.
- **Inspección de /home/LogFiles/:** Permite revisar los archivos `_docker.log` antes de la rotación automática del sistema de archivos PaaS.

AZURE MONITOR, LOG ANALYTICS Y CATEGORÍAS KQL

- **3 Categorías Oficiales:**
  - `AppServiceConsoleLogs`: Salida de `stdout` y `stderr` del contenedor.
  - `AppServiceHTTPLogs`: Trazas de peticiones HTTP, códigos y tiempos de respuesta.
  - `AppServicePlatformLogs`: Eventos de orquestación y ciclo de vida PaaS.
- **Consulta KQL de Excepciones:**

```
AppServiceConsoleLogs | where ResultDescription has "ERROR" | project TimeGenerated, ResultDescription
```

Diagrama de Arquitectura Lógica de la Solución

Caso de uso empresarial de procesamiento inteligente de documentos, desacoplamiento PaaS y autenticación Zero-Trust

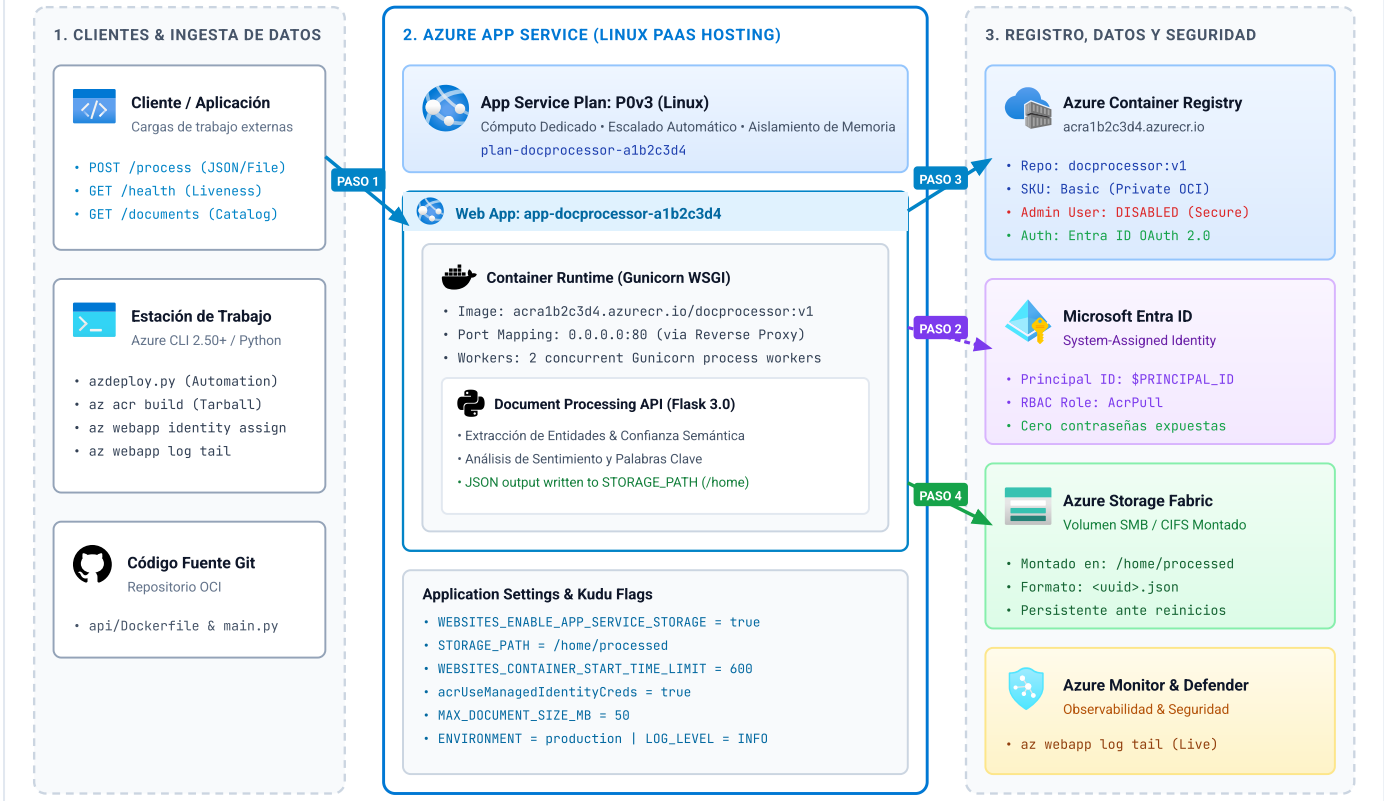
CASO DE USO REAL: INGESTA Y ANÁLISIS SEGURO DE DOCUMENTOS LEGALES Y FINANCIEROS

Problemática Empresarial: Una entidad corporativa procesa diariamente contratos confidenciales y reportes financieros mediante modelos de IA. Anteriormente utilizaban servidores locales con credenciales maestras compartidas en texto plano, sin persistencia ante fallos y con caídas frecuentes durante picos de carga.

Solución de Arquitectura en Azure: Migración a un microservicio en contenedor alojado en **Azure App Service en Linux**. La imagen se almacena en **Azure Container Registry (ACR)** y el contenedor se autentica de forma segura mediante **System-Assigned Managed Identity** con el rol `AcrPull`, sin exponer contraseñas. Los resultados analíticos se almacenan en el volumen persistente `/home/processed` respaldado por Azure Storage.

BLUEPRINT DE ARQUITECTURA LÓGICA (DRAW.IO CANVAS ESTILO VECTORIAL)

Zero-Trust & Persistent PaaS



Secuencia Lógica: (1) Ingesta: Peticiones HTTPS entrantes balanceadas al contenedor Flask en el puerto 80. **(2) Autenticación Zero-Trust:** App Service presenta el token de Entra ID vía Managed Identity sin credenciales fijas. **(3) Descarga OCI:** Kudu extrae las capas de `docprocessor:v1` desde ACR con permisos `AcrPull`. **(4) Persistencia Segura:** Los análisis procesados se escriben en `/home/processed` respaldado por Azure Storage resiliente.

Diagrama de Arquitectura Física y Red de App Service

Topología de red perimetral, plano de datos aislado, servicio IMDS y montaje CIFS/SMB cifrado

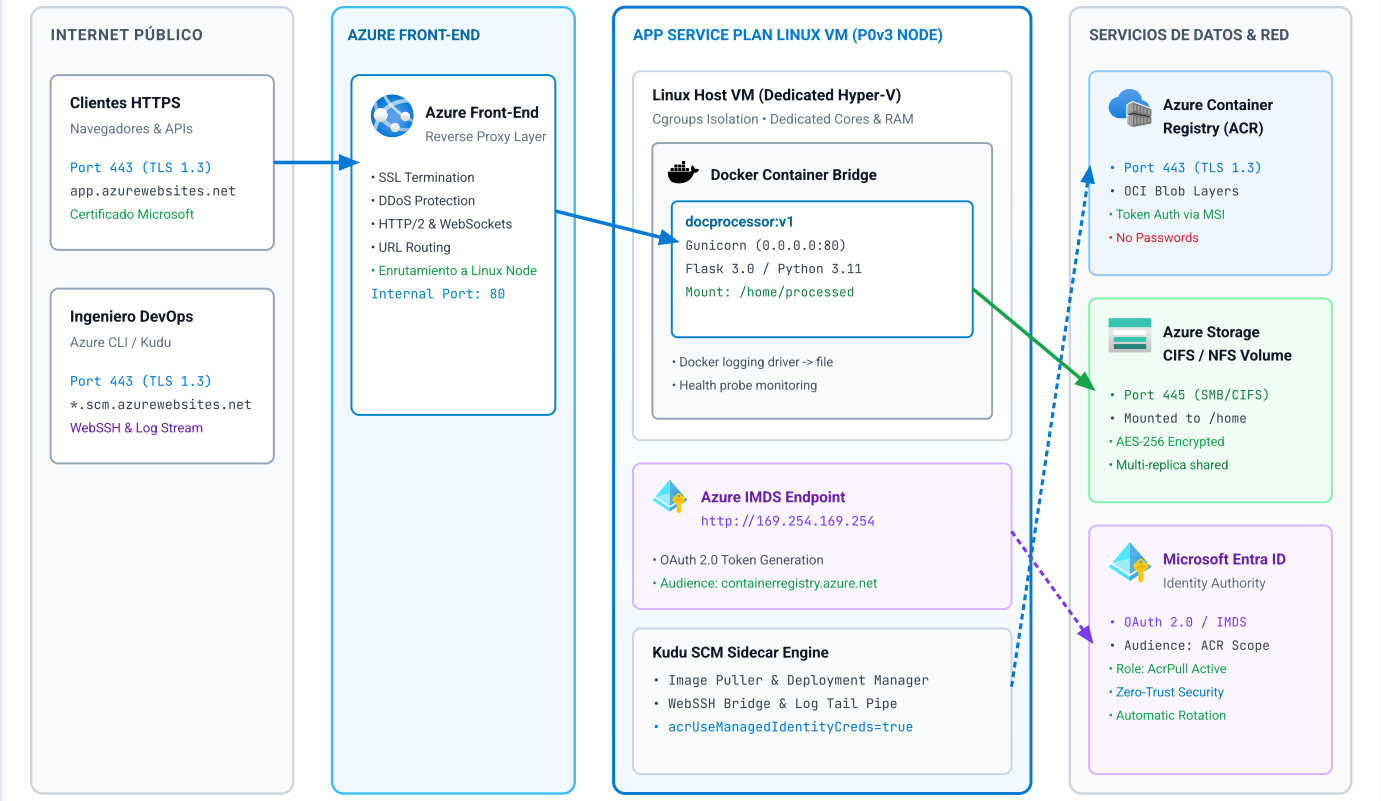
TOPOLOGÍA DE RED Y FRONTERAS DE AISLAMIENTO FÍSICO

Aislamiento de Cómputo: El App Service Plan Linux aprovisiona instancias de máquinas virtuales dedicadas protegidas por Azure Hyper-V y cgroups. El tráfico de producción ingress en el puerto 443 es terminado en el balanceador frontal PaaS de Azure y reenviado por un canal local al contenedor Docker en `0.0.0.0:80`.

Autenticación Criptográfica IMDS: La identidad administrada consulta internamente al Azure Instance Metadata Service (`http://169.254.169.254/metadata/identity/oauth2/token`), una dirección local no enrutable a Internet. Los tokens obtenidos son enviados por TLS 1.3 hacia el plano de datos de Azure Container Registry.

TOPOLOGÍA DE RED Y PLANO FÍSICO (DRAW.IO CANVAS ESTILO VECTORIAL)

TLS 1.3 • IMDS 169.254.169.254 • SMB Port 445



Aspectos Clave de Seguridad de Red: (1) Terminación TLS: El balanceador PaaS frontal de Azure gestiona certificados SSL y mitiga ataques de capa 7 antes de alcanzar el nodo Linux. **(2) Cero Tráfico de Credenciales:** El daemon IMDS entrega tokens en memoria sin exponer secretos en el canal de red público. **(3) Montaje de Red Cifrado:** El almacenamiento en `/home` se comunica por canal CIFS interno cifrado con AES-256 gestionado por Microsoft.

Limpieza de Recursos y Diagnóstico Oficial (Troubleshooting)

Desaprovisionamiento asíncrono, resolución de incidencias del laboratorio y reglas de oro de arquitectura

Fase de Limpieza de Recursos (Clean up resources)

`az group delete (--no-wait --yes)` **Clean Up**

Finalizado el ejercicio, elimine el grupo de recursos para evitar costos continuos de cómputo dedicado:

```
# Borrado seguro y asíncrono del Resource Group y todos los recursos creados
az group delete --name <your-resource-group-name> --no-wait --yes
```

⚠ **Precaución de Limpieza en Cascada:** El comando `az group delete` elimina de forma permanente el grupo de recursos y todos los componentes asociados (App Service Plan, Web App, registros de ACR y almacenamiento montado). Si utilizó un grupo existente con recursos ajenos al ejercicio, éstos también se eliminarán.

Guía Oficial de Resolución de Problemas (Troubleshooting de Microsoft Learn)

1. AUTENTICACIÓN Y VARIABLES DE SESIÓN

• **Verificar suscripción:** Ejecute `az account show` para confirmar que opera en la suscripción activa correcta.
• **Variable vacía:** Si `$env:ACR_NAME` está vacía en una terminal nueva, reejecute: `. .\.env.ps1` (PowerShell) o `source .env` (Bash).

2. VERIFICACIÓN DE DESPLIEGUE DEL ACR

• **Portal de Azure:** Localice su grupo de recursos y confirme que ACR muestre *Provisioning State* en **Succeeded**.
• **Comprobación CLI:** Ejecute `az acr list --output table` para validar que el registro es accesible.

3. FALLAS DE COMPILACIÓN EN ACR TASKS

• **Directorio raíz:** Confirme que ejecuta el comando desde la raíz del proyecto (donde reside la carpeta `api/`).
• **Logs de tarea:** Liste ejecuciones con `az acr task list-runs --registry $env:ACR_NAME -o table` e inspeccione con `az acr task logs --run-id <id>`.

4. ERRORES 403 / IMAGEPULLBACKOFF

• **Identidad Administrada:** Compruebe que la Web App tenga la identidad activa con `az webapp identity show`.
• **Rol AcrPull:** Confirme la asignación RBAC. La propagación en Entra ID puede tardar 1-2 minutos. Reejecute la configuración del contenedor.

5. ERRORES 502/503 Y STARTUP TIMEOUT

• **Descarga inicial:** La primera petición tarda más mientras App Service descarga las capas de la imagen.
• **Logs en vivo:** Transmita logs con `az webapp log tail`. Active `always-on true` y amplíe `WEBSITES_CONTAINER_START_TIME_LIMIT=600`.

6. VERIFICACIÓN DE ALMACENAMIENTO PERSISTENTE

• **App Setting:** Verifique que `WEBSITES_ENABLE_APP_SERVICE_STORAGE=true` figure en la configuración.
• **Endpoint /documents:** Llame a `/documents` tras enviar un documento para comprobar que el archivo JSON persiste en el volumen.

7. CONFLICTOS DE PUERTO (WEBSITES_PORT VS EXPOSE)

• **Puerto no estándar:** Si la app escucha en el puerto 8000 o 5000, defina `WEBSITES_PORT=8000`. Modificar el Dockerfile con `EXPOSE 80` no altera el socket del proceso.
• **Enlace de red:** El servidor debe enlazar obligatoriamente a `0.0.0.0` y no a `127.0.0.1`.

8. FALLAS DE KEY VAULT Y STARTUP COMMAND

• **Key Vault RBAC:** Si `@Microsoft.KeyVault(...)` falla, asigne el rol `Key Vault Secrets User` a la Managed Identity.
• **Sintaxis de inicio:** Errores tipográficos en `--startup-file` impiden que el contenedor complete el arranque.

Reglas de Oro de Arquitectura para el Examen AI-200 (PPT y Microsoft Learn)

REGLA 1: AUTENTICACIÓN ZERO-TRUST

Nunca active credenciales de administrador en el ACR para producción. Asigne **System-Assigned Managed Identity** a la Web App con el rol `AcrPull` de mínimo privilegio en el ámbito del registro.

REGLA 2: PERSISTENCIA CIFS EN /HOME

El almacenamiento de contenedores Linux en App Service es efímero. Active `WEBSITES_ENABLE_APP_SERVICE_STORAGE=true` para montar almacenamiento persistente cifrado con AES-256 sobre Azure Storage.

REGLA 3: MITIGACIÓN DE COLD START

Para microservicios de IA, configure `always-on true` y amplíe `WEBSITES_CONTAINER_START_TIME_LIMIT=600` para evitar que App Service termine el contenedor durante la carga pesada del modelo.

REGLA 4: AISLAMIENTO DEL PLANO SCM/KUDU

El motor de administración y despliegue corre en un contenedor aislado accesible vía `<app>.scm.azurewebsites.net`, permitiendo inspeccionar logs y ejecutar SSH sin exponer la app productiva.

REGLA 5: INMUTABILIDAD Y DIGESTS SHA-256 (SLIDE 12)

En producción, use digests inmutables (`@sha256:...`) en lugar de tags mutables (`:v1`), y active `--write-enabled false` en ACR para prevenir alteraciones de imágenes.

REGLA 6: ADHESIÓN A RANURA / SLOT SETTINGS (SLIDE 18 & 21)

• **Evitar fugas en Slot Swap:** Variables dependientes del entorno (ej. `API_ENDPOINT`, connection strings de BD) deben marcarse con *Deployment slot setting* (CLI: `--slot-settings`) para anclarse físicamente a su ranura.
• **Comando CLI de Swap:** `az webapp deployment slot swap -g $RG -n $APP_NAME --slot staging --target-slot production` conmuta el tráfico VIP con cero downtime.

🎉 **Felicitaciones:** Ha completado exitosamente el laboratorio *Deploy a container to Azure App Service* de Microsoft Learn.

Preguntas Oficiales de Examen de Certificación

Knowledge check oficial de Microsoft Learn y distribución de habilidades evaluadas en el Examen AI-200

DISTRIBUCIÓN DE HABILIDADES EVALUADAS EN EL EXAMEN OFICIAL AI-200 (GUÍA DE ESTUDIO MICROSOFT LEARN)

AI-200 Study Guide

1. Contenedores Azure — 20% – 25%	2. Datos de IA en Azure — 25% – 30%	3. Conexión de Servicios — 20% – 25%	4. Seguridad y Monitoreo — 20% – 25%
★ **Evaluado en Lab 2:** App Service Linux, Managed Identity, AcrPull, persistencia /home y Kudu.	Cosmos DB NoSQL, PostgreSQL con `pgvector`, caché semántica y similitud en Managed Redis.	Mensajería con Service Bus y Event Grid, microservicios dirigidos por eventos en Azure Functions.	Identidades Entra ID, Key Vault, App Configuration, observabilidad OpenTelemetry y consultas KQL.

Pregunta 1: Resolución de Conflictos de Puerto y Errores de Conexión (Slide 21 Oficial)

Runtime Port Mapping & Reverse Proxy

Una imagen de contenedor escucha en el puerto `8000`. Después de la implementación en App Service, las solicitudes devuelven errores de conexión. ¿Qué resuelve este problema?

A) Modificar el Dockerfile para usar `EXPOSE 80`.
B) Establecer la configuración de la aplicación `WEBSITES_PORT` en `8000`.
C) Habilitar el protocolo HTTP/2 en la configuración de la plataforma.

✓ **Respuesta Correcta: B) Establecer la configuración de la aplicación WEBSITES_PORT en 8000**

- **Por qué WEBSITES_PORT=8000:**
  - Instruye al proxy inverso PaaS hacia qué puerto privado interno debe reenviar el tráfico entrante.
- **Por qué falla EXPOSE 80 (Distractor A):**
  - `EXPOSE` en Docker es solo metadato documental; no cambia el socket TCP que el proceso de la app abre en runtime (sigue en 8000).
- **Regla de Oro de Arquitectura:**
  - Cualquier puerto distinto de 80/8080 exige `WEBSITES_PORT`. En producción, incluso con puerto 80, configurarlo explícitamente suprime la latencia de autodetección de Azure.

Flujo de Tráfico y Mapeo de Puertos (Exterior vs Interior)

1. **Internet / Clientes Externos** — HTTPS :443 / HTTP :80
   URL pública estándar: `https://<app>.azurewebsites.net`
   ▼
2. **App Service Frontend (Reverse Proxy)** — Enrutador PaaS
   ✕ **Sin config:** Busca puerto 80/8080 → *Connection Refused*
   ✓ **Con WEBSITES_PORT=8000:** Reenvía tráfico directo a :8000
   ▼
3. **Contenedor Docker (VM PaaS)** — Socket TCP :8000
   Proceso activo: `uvicorn main:app --host 0.0.0.0 --port 8000`

⚠ **Trampa de Examen:** `EXPOSE 80` en Dockerfile solo actúa como metadato documental. No reconfigura el socket interno de la aplicación.

Pregunta 2: Ranuras de Implementación

Slot Sticky Settings

Una app requiere distintas URLs de API para ensayo (staging) y producción. ¿Cómo impedir que la URL de ensayo cambie a producción durante un swap?

A) Almacenar el endpoint de API en la imagen para cada entorno.
B) Establecer `API_ENDPOINT` como configuración de ranura (slot setting).
C) Usar cadenas de conexión en lugar de App Settings.

✓ **Respuesta Correcta: B) Configuración de ranura (slot setting)**

Justificación: Marcar una variable como *slot setting* asegura que permanezca anclada a la ranura física específica durante operaciones de intercambio (slot swap).

Pregunta 3: Autenticación Zero-Trust

Managed Identity & RBAC

Se debe extraer una imagen desde un ACR privado sin contraseñas estáticas y con el principio de mínimos privilegios. ¿Qué combinación cumple este requisito?

A) Habilitar cuenta admin en ACR y configurar contraseñas.
B) Managed Identity en Web App, rol `AcrPull` en ACR y `acrUseManagedIdentityCreds=true`.
C) Rol `Contributor` al App Service Plan en la suscripción.

✓ **Respuesta Correcta: B) Managed Identity + rol AcrPull en ACR**

Justificación: Managed Identity elimina contraseñas. El rol `AcrPull` confiere permiso exclusivo de lectura sobre el registro OCI sin permisos de escritura ni borrado.

Pregunta 4: Persistencia de Datos

Almacenamiento Persistente

Una API guarda PDFs en `/home/processed`. Tras un reinicio, los archivos desaparecen. ¿Qué configuración resuelve el problema?

A) Configurar `WEBSITES_ENABLE_APP_SERVICE_STORAGE=true`.
B) Escalar el App Service Plan a nivel P3v3 con más RAM.
C) Cambiar la ruta de trabajo de `/home/processed` a `/tmp`.

✓ **Respuesta Correcta: A) WEBSITES_ENABLE_APP_SERVICE_STORAGE=true**

Justificación: El almacenamiento es efímero por defecto. La variable monta un volumen CIFS persistente respaldado por Azure Storage en `/home`.

Pregunta 5: Timeout en el Arranque

Startup Probe Tuning

Un contenedor tarda 6 minutos en cargar modelos. App Service aborta el proceso por timeout a los 230s. ¿Qué configuración soluciona el fallo?

A) Configurar `WEBSITES_CONTAINER_START_TIME_LIMIT=600`.
B) Cambiar el protocolo de la aplicación de HTTP a gRPC.
C) Forzar el puerto en el Dockerfile usando `EXPOSE 443`.

✓ **Respuesta Correcta: A) WEBSITES_CONTAINER_START_TIME_LIMIT=600**

Justificación: El timeout por defecto es de 230s. La variable permite ampliarlo hasta 1800s (30 min) para cargas pesadas de IA.

Pregunta 6: Restricciones de Cómputo (SKU)

SKU & Linux Requirements

Un desarrollador intenta crear una Web App para contenedores usando un App Service Plan en el SKU Free (F1) y la CLI rechaza la creación. ¿Qué requisito causa el fallo?

A) Contenedores personalizados en Linux requieren nivel dedicado (Basic B1 o superior / Premium).
B) El SKU Free solo admite contenedores compilados en arquitecturas ARM64.
C) Los contenedores requieren obligatoriamente una VNet con ExpressRoute.

✓ **Respuesta Correcta: A) Requiere plan dedicado (Basic B1 en adelante o Premium)**

Justificación: Web App for Containers en Linux no está soportada en niveles compartidos (Free F1 o Shared D1). Requiere cómputo dedicado (Basic B1 o Premium).

Pregunta 7: Transmisión en Vivo de Logs

Observabilidad & Logging

Se requiere depurar en tiempo real las excepciones no capturadas de una API en contenedor. ¿Qué comando de Azure CLI transmite inmediatamente stdout/stderr a la consola?

A) `az webapp log tail --name $APP_NAME --resource-group $RESOURCE_GROUP`
B) `az webapp deployment source show --name $APP_NAME`
C) `az monitor metrics list --resource $APP_NAME`

✓ **Respuesta Correcta: A) az webapp log tail**

Justificación: `az webapp log tail` abre un túnel websocket contra el endpoint SCM/Kudu, transmitiendo en vivo stdout/stderr de los workers y del motor Docker.

SIMULACIÓN INTERACTIVA DE EVALUACIÓN CON MICROSOFT COPILOT CHAT (SLIDE 22 OFICIAL)

Prompt Oficial de Examen para Copilot Chat:

```
"As an Azure App Service expert, please create three multiple-choice questions that assess knowledge of the container lifecycle on App Service. Present one question at a time and wait for my response before continuing. After each answer, provide a brief explanation of the correct choice before moving on to the next question."
```

Instrucción pedagógica: Ejecute este prompt en Microsoft Copilot o Microsoft 365 Chat para afianzar los conceptos del ciclo de vida de contenedores antes de rendir el examen oficial AI-200.

Glosario de Términos, Conceptos Clave y Siglas de Azure

Definiciones normativas de Azure App Service, diccionario de siglas y mapa curricular oficial

1. Conceptos Arquitectónicos Centrales de Azure App Service

APP SERVICE PLAN (ASP)
Unidad de cómputo físico/virtual que determina la región, el número de instancias, el SO (Linux/Windows), vCPUs y memoria RAM asignados a una o varias Web Apps.

WEB APP FOR CONTAINERS
Servicio PaaS administrado que ejecuta contenedores Docker/OCI en Linux con balanceo de carga integrado, certificados SSL gratuitos y escalado automático.

SYSTEM-ASSIGNED IDENTITY
Identidad administrada ligada al ciclo de vida del recurso en Microsoft Entra ID. Proporciona autenticación sin contraseñas para interactuar con ACR y Key Vault.

ALMACENAMIENTO PERSISTENTE /HOME
Volumen de red compartido (SMB/CIFS) respaldado por Azure Storage montado en `/home` cuando `WEBSITES_ENABLE_APP_SERVICE_STORAGE=true`.

KUDU / SCM ENGINE
Plano de control y motor de gestión de App Service accesible en `<app>.scm.azurewebsites.net`, encargado de pulls de imagen, WebSSH y telemetría de log.

CONTAINER LOGGING STREAM
Mecanismo de captura y retransmisión en tiempo real de los flujos `stdout` y `stderr` del contenedor Docker hacia Azure CLI y Azure Monitor.

DEPLOYMENT SLOT (RANURA)
Entorno de hospedaje secundario aislado con FQDN propio para pruebas previas al pase a producción, con soporte de variables pegadas (slot settings) que no se intercambian.

KEY VAULT REFERENCE
Directiva declarativa `@Microsoft.KeyVault(...)` que resuelve secretos en memoria al iniciar el contenedor mediante Managed Identity sin librerías en el código.

STARTUP COMMAND OVERRIDE
Instrucción en App Service que reemplaza la directiva `CMD` del Dockerfile (manteniendo `ENTRYPOINT`) para ejecutar migraciones o argumentos dinámicos.

2. Diccionario de Siglas y Acrónimos Técnicos del Módulo

Sigla	Significado y Rol en Azure
AAS	**Azure App Service:** Plataforma PaaS administrada para hospedar aplicaciones web y contenedores.
ASP	**App Service Plan:** Asignación de recursos de proceso (CPU, memoria, SKU y escala) para Web Apps.
ACR	**Azure Container Registry:** Registro privado administrado para almacenamiento y compilación OCI.
RBAC	**Role-Based Access Control:** Control de acceso basado en roles (`AcrPull`, `Contributor`).
MI / MSI	**Managed Identity / Managed Service Identity:** Identidades sin contraseña gestionadas por Entra ID.
OCI	**Open Container Initiative:** Estándar de la industria para formatos de imagen y runtimes de contenedores.
KUDU	**Kudu Engine:** Motor de despliegue, administración y diagnóstico asociado a cada Web App.
SCM	**Site Control Manager:** Endpoint administrativo aislado en `<app>.scm.azurewebsites.net`.
FQDN	**Fully Qualified Domain Name:** Nombre de dominio completo (ej. `<app>.azurewebsites.net`).
CMD	**Command:** Directiva de inicio en Dockerfile sobreescribible con el parámetro startup-file.
KQL	**Kusto Query Language:** Lenguaje de consulta analítica para telemetría en Log Analytics.

Sigla	Significado y Rol en Azure
TLS	**Transport Layer Security:** Protocolo criptográfico (TLS 1.3) que cifra el canal HTTPS de la Web App.
CIFS / SMB	**Common Internet File System:** Protocolo de red usado para montar `/home` desde Azure Storage.
IMDS	**Instance Metadata Service:** Endpoint local `169.254.169.254` para emisión de tokens Entra ID.
SKU	**Stock Keeping Unit:** Nivel de tarificación y capacidad de cómputo (Basic B1, Premium P0v3).
WSGI	**Web Server Gateway Interface:** Interfaz estándar entre servidores web (Gunicorn) y Python Flask.
REST	**Representational State Transfer:** Arquitectura de servicios web sin estado sobre protocolo HTTP.
CORS	**Cross-Origin Resource Sharing:** Mecanismo de seguridad para autorizar peticiones entre dominios.
ARM	**Azure Resource Manager:** Plano de control declarativo para gestionar infraestructura en Azure.
CVE	**Common Vulnerabilities and Exposures:** Identificador público estandarizado de fallos de seguridad.
DNS	**Domain Name System:** Servicio de resolución de nombres con ámbito global único (FQDN).
MIME	**Multipurpose Internet Mail Extensions:** Formato estándar de tipos de contenido HTTP (API REST).

3. Mapa Curricular y Rutas de Aprendizaje Oficiales de Microsoft Learn (Curso AI-200T00-A)

Curso Oficial Microsoft: AI-200T00-A: Desarrollo de soluciones en la nube de IA en Azure (5 días / 120 horas lectivas)

Certificación Asociada: Microsoft Certified: Azure AI Cloud Developer Associate (Examen AI-200)

FASE 1: CÓMPUTO & CONTENEDORES (DOM. 1)
- **Módulo 1:** Hospedaje de contenedores en Azure (ACR, ACR Tasks serverless, OCI).
- **Módulo 2:** Implementación en Azure App Service (Linux PaaS, Managed Identity). **[★ Este Lab]**
- **Módulo 3:** Implementación y escalado con KEDA en Azure Container Apps (ACA).

FASE 2: DATOS & EMBEDDINGS (DOM. 2)
- **Módulo 4:** Soluciones de IA con Azure Cosmos DB for NoSQL y vector search.
- **Módulo 5:** Bases de datos vectoriales con Azure Database for PostgreSQL (`pgvector`).
- **Módulo 6:** Optimización de latencia y caché semántica con Azure Managed Redis.

FASE 3: INTEGRACIÓN & DEVOPS (DOM. 3 Y 4)
- **Módulo 7:** Integración desacoplada con Azure Service Bus, Event Grid y Functions.
- **Módulo 8:** Gestión de secretos y feature flags en Azure Key Vault y App Configuration.
- **Módulo 9:** Observabilidad de extremo a extremo, OpenTelemetry y diagnóstico KQL.

Microsoft Learn Study Guide: https://learn.microsoft.com/es-es/credentials/certifications/resources/study-guides/ai-200

Microsoft Worldwide Learning (WWL) • Curso AI-200T00-A